

Internship Report

On

Object Detection Using Python

Submitted To
School of Computer Science and Engineering IILM
University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE(AI/ML)



By

Roll Number of Student: CS-2341620
Name of Student: RAVIKANT MISHRA
Section & Semester: 4cse6, 7th Semester
Organization: Eduskills Academy
Batch: 2026-27

1. Candidate's Declaration

I, RAVIKANT MISHRA, do hereby declare that the Internship report titled "Object Detection Using Python" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own work and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

This report documents the design, development, and implementation of the Object Detection Using Python project using Python, YOLOv8, OpenCV, Tkinter and related computer vision technologies.

Signature: _____

Student Name: RAVIKANT MISHRA Roll

No.: CS-2341620

2. Acknowledgement

I would like to express my sincere gratitude to Edu skills Academy for providing me the opportunity and academic environment to undertake this internship in the IT Department. I am thankful to my industry Mentor, Mrs. Parul Saini, for her continuous guidance, technical support, valuable feedback and encouragement throughout the development of this project.

I am using this opportunity to express my gratitude to everyone who supported me throughout the project. I am thankful for their aspiring guidance, constructive criticism, and friendly advice during this work.

I also extend my gratitude to the faculty of IILM University, Greater Noida, for their academic support and for facilitating this project as part of my Computer Science and Engineering curriculum. I express my gratitude to my parents and friends for their blessings and support.

Signature: _____

Student Name: RAVIKANT MISHRA

RollNo: CS-2341620

3. Supporting Certificate

The following certificate is attached as supporting documentation. It certifies that Ravikant Mishra, IILM University, Greater Noida, successfully completed an 8-weeks Python Fullstack Developer Virtual Internship during June–August 2026 under the EduSkills Academy virtual internship programme.



Figure 3.1 — Certificate of Virtual Internship (EduSkills Academy / AICTE Virtual Internship)

Table of Contents

1. Candidate's Declaration	1
2. Acknowledgement	2
3. Supporting Certificate	3
4. Project Description	5
4.1 Introduction	5
4.2 Project Background	6
4.3 Problem Statement	6
4.4 Project Objectives	6
4.5 Scope of the Project	6
4.6 Technologies and Tools Used	7
4.7 System Architecture	7
4.7.1 Detection Pipeline	7
4.7.2 Data Flow and Functional Components	8
4.7.3 Entity-Relationship Model	8
4.7.4 Use Case Model	8
4.8 Methodology	9
4.8.1 Implementation	10
4.8.2 Testing and Demonstration	10
4.8.3 Challenges Faced	10
4.9 Expected Outcomes	13
4.9.1 Future Scope	14
4.9.2 Conclusion	14
4.10 References	14

4. Project Description

4.1 Introduction

Object detection is a fundamental computer vision task concerned with both identifying objects in an image or video frame and locating them using bounding boxes. Unlike image classification, which assigns a label to an entire image, object detection can identify multiple objects and indicate where each object occurs. The project material identifies applications such as autonomous driving, surveillance, facial recognition, medical imaging and smart retail.

Python is well suited to this domain because of its readable syntax and its extensive ecosystem for image processing and deep learning. OpenCV provides image and video processing capabilities, while modern object detection models such as YOLO, SSD and Faster R-CNN can be integrated through pre-trained weights and APIs.

The project implements a real-time object detection application using a YOLOv8 model with OpenCV and a Tkinter-based graphical interface. The application can display detections, capture snapshots and record annotated video frames.

4.2 Project Background

The project is based on the rapid development of deep learning based object detection. Traditional image processing approaches have increasingly been supplemented or replaced by convolutional neural network based detectors that can learn visual features and predict object classes and locations. The supplied project report discusses single-shot detectors, transfer learning, real-time inference and lightweight models as important directions in the field.

The practical implementation described in the source project uses the YOLOv8 nano model through the Ultralytics library. A webcam frame is captured with OpenCV, resized, passed to the model for inference, annotated with detection results, converted for Tkinter display, and optionally written to a video file.

4.3 Problem Statement

The project aims to develop and implement an efficient and scalable object detection system using Python, state-of-the-art deep learning models and libraries such as Tkinter, Ultralytics and OpenCV. The system should identify and classify multiple objects in real time with useful accuracy across diverse environments and should remain adaptable to different deployment environments.

4.4 Project Objectives

- To develop a Python-based object detection application for real-time video input.
- To use YOLOv8 for object localization and classification.
- To integrate OpenCV for camera capture, image processing and video recording.
- To provide a simple Tkinter graphical interface for operating the detector.
- To display object labels and bounding boxes on detected objects.
- To support snapshot capture and recording of annotated detection output.
- To study the practical limitations and future directions of real-time object detection.

4.5 Scope of the Project

The scope covers image/video acquisition, preprocessing, object detection inference, post-processing, visualization and optional saving of results. The application is primarily demonstrated using a webcam and the YOLOv8 nano model. The conceptual scope also includes comparison with Faster R-CNN, SSD and lightweight MobileNet-SSD approaches, although the supplied implementation specifically demonstrates YOLOv8.

- Real-time webcam-based object detection.
- Multiple-object and multi-class detection where supported by the model.

- Bounding-box visualization and class labels.
- Snapshot and video recording functionality.
- A desktop GUI based on Tkinter.
- Possible future extension to edge, mobile and application-specific deployments.

4.6 Technologies and Tools Used

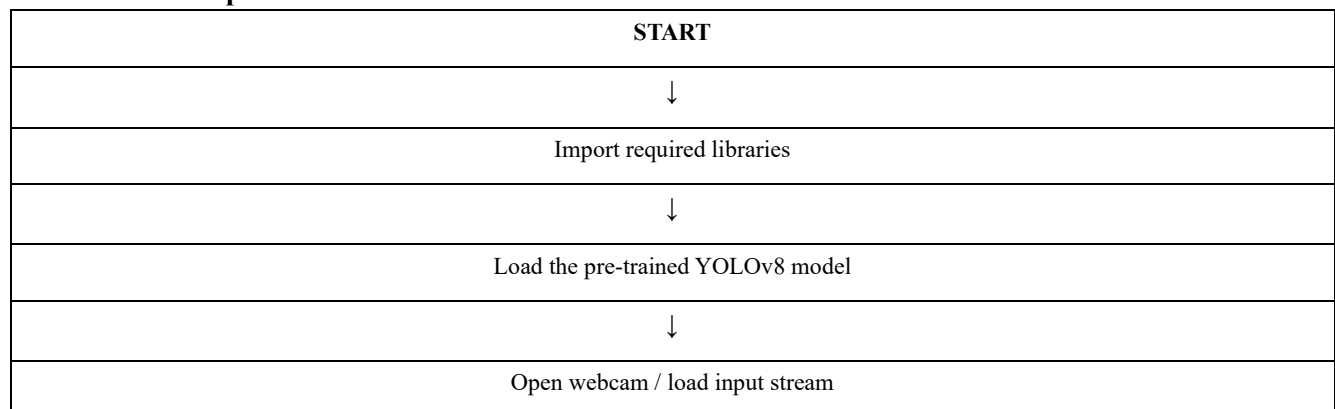
Technology / Tool	Role in the Project	Use
Python	Core programming language	Application logic and integration
YOLOv8 / Ultralytics	Object detection model	Inference and object recognition
OpenCV	Computer vision	Camera capture, resize, color conversion, video writing
Tkinter	GUI framework	Desktop user interface and controls
Pillow (PIL)	Image handling	Convert frames for Tkinter display
Threading	Execution support	Run detection without blocking the GUI
Time	Utility	Timestamped snapshot filenames

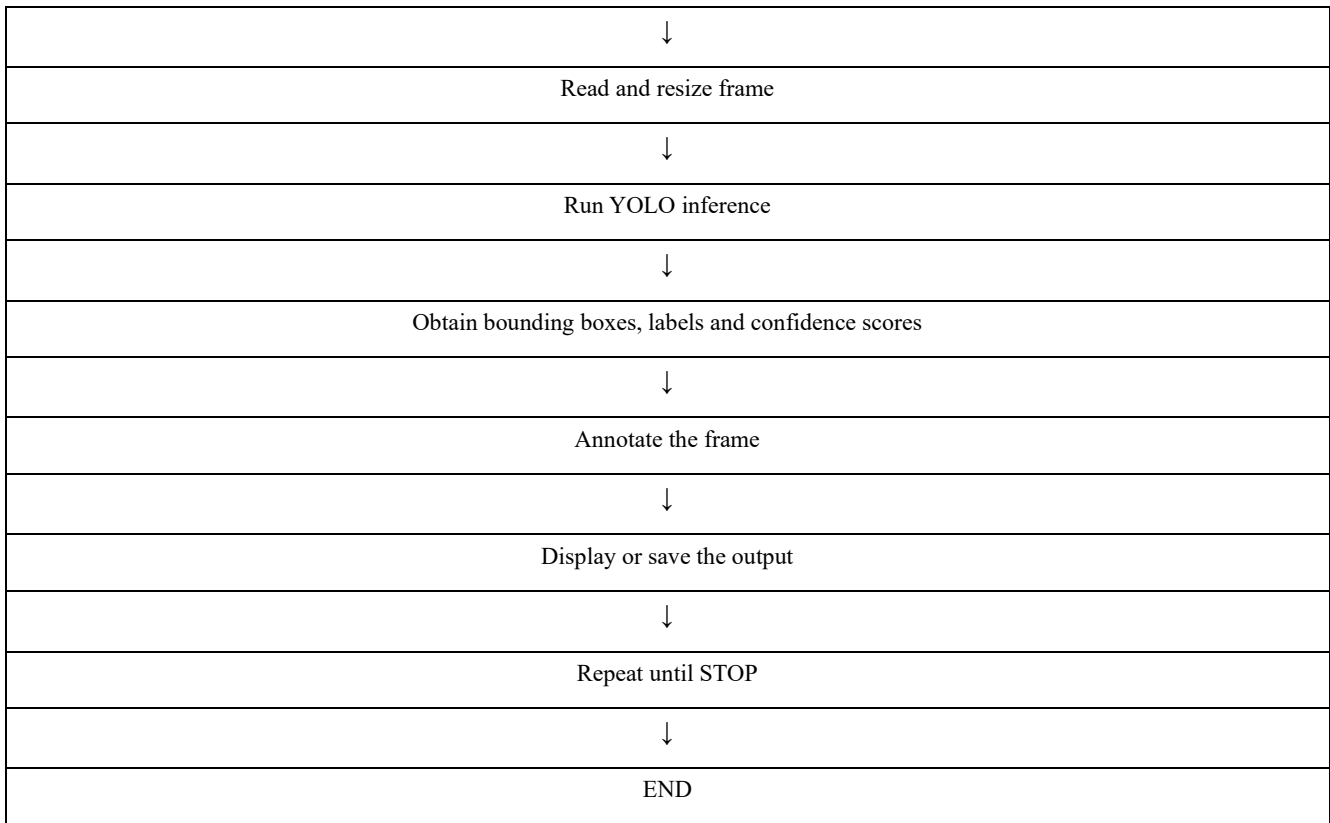
4.7 System Architecture

The system is organized as a simple processing pipeline. Input is obtained from a camera or image/video source, the frame is prepared for inference, the YOLOv8 model produces detection results, and the results are visualized as bounding boxes and class labels. The application then displays the annotated frame and can save snapshots or recorded video.

Input	Preprocessing	Inference	Post-processing	Output
Webcam / Image / Video				
	Resize & frame preparation			
		YOLOv8 model		
			Bounding boxes + labels + confidence	
				GUI display / snapshot / video

4.7.1 Detection Pipeline





4.7.2 Data Flow and Functional Components



Figure 4.7.1 — Data Flow Diagram: Input Image → Object Detection → Recognition

- Capture Input: the user can provide an image/video or a camera can provide a real-time video feed.
- Preprocess Image/Video: the input is resized to match the model requirements and prepared for inference.
- Run Detection Model: the pre-processed frame is passed to the object detection model, which generates bounding boxes, labels and confidence scores.
- Post-Processing: redundant or low-quality detections can be filtered so that the most useful bounding boxes are retained.
- Display/Save Results: the final annotated output is displayed to the user or saved as an image/video.

4.7.3 Entity-Relationship Model

The conceptual data model described in the project report contains three main entities: Image, DetectionModel and ObjectDetected. Although the demonstrated application is primarily an in-memory real-time desktop application, this model can be used if detection results are later persisted to a database.

Entity	Attributes	Relationship
Image	image_id (PK), file_path, upload_time	One image can have many detected objects
DetectionModel	model_id (PK), model_name, framework, version	One model can detect many objects

ObjectDetected	detection_id (PK), object_label, confidence_score, bounding_box	Links an image and a detection model
----------------	--	--------------------------------------

4.7.4 Use Case Model

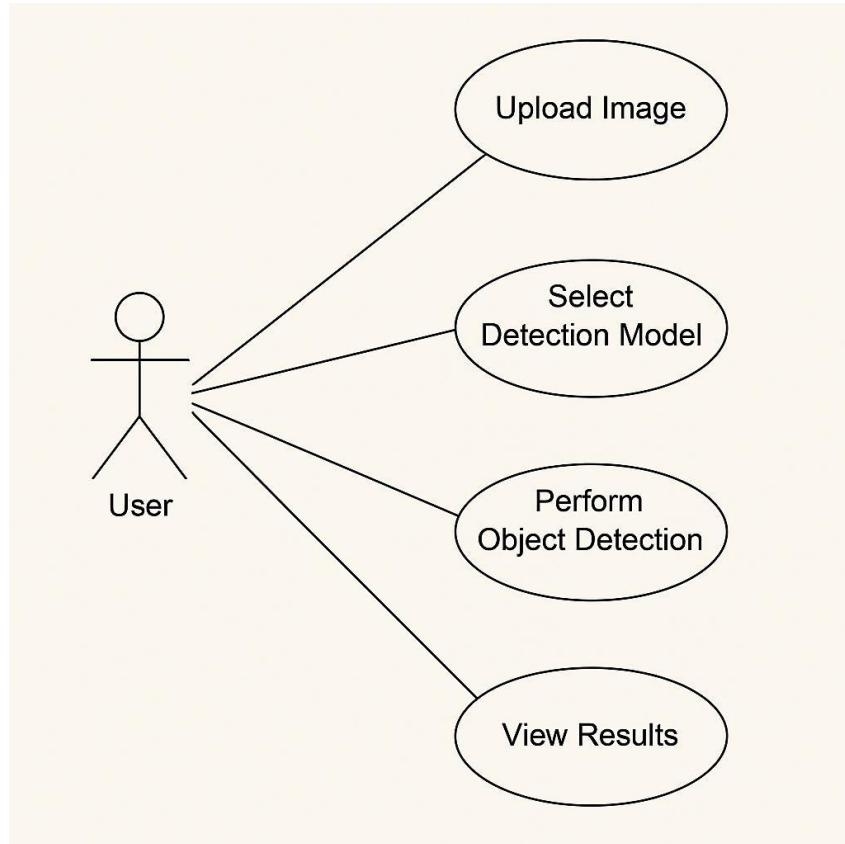


Figure 4.7.2 — Use Case Diagram for the Object Detection Application

- Upload / provide input image or video.
- Select or initialize the detection model.
- Start object detection.
- View the annotated detection results.
- Capture a snapshot or record the detection output.

4.8 Methodology

Development followed an iterative build-and-test approach. The workflow began with understanding the object detection problem and selecting a practical pre-trained model. The implementation then integrated the model with OpenCV and Tkinter, followed by testing the live camera pipeline and output functions.

4.8.1 Model and Inference Method

The supplied implementation loads the YOLOv8 nano model using the Ultralytics Python interface. For every captured frame, the program resizes the image to 640×480 pixels and passes it to the model. The returned result is plotted onto the frame to create an annotated image.

4.8.2 GUI and Interaction Method

The graphical interface is implemented with Tkinter. It provides a title area, video display area and control buttons for starting detection, stopping detection, capturing snapshots and starting/stopping recording. A background thread is used for the detection operation while Tkinter's scheduling mechanism updates the displayed frame.

4.8.3 Recording and Snapshot Method

When recording is enabled, OpenCV's VideoWriter is initialized using the XVID codec and the annotated frames are written to an AVI file. Snapshot capture saves the latest annotated frame as a JPG file using a timestamp in the filename.

4.8.4 Implementation

```
import tkinter as tk from tkinter
import messagebox from PIL
import Image, ImageTk import
threading import cv2 from
ultralytics import YOLO import
time

class YOLOv8App:
def __init__(self, root):
    self.root = root
    self.root.title("Object Detection")
    self.root.geometry("850x650")
    self.model = YOLO("yolov8n.pt")
    self.cap = None    self.running =
False    self.recording = False
    self.writer = None    self.last_frame
= None

    def start_detection(self):    self.running = True
    self.cap = cv2.VideoCapture(0)
    threading.Thread(target=self.detect_objects).start()

    def stop_detection(self):
    self.running = False    if
    self.cap:
        self.cap.release()
    if self.writer:
        self.writer.release()

    def capture_snapshot(self):
    if self.last_frame is not None:
        filename = f"yolo_snapshot_{int(time.time())}.jpg"
        cv2.imwrite(filename, self.last_frame)

    def detect_objects(self):
        # Capture, resize, infer, annotate and display frames
    pass
```

Figure 4.8.1 — Representative implementation excerpt based on the supplied project code

4.8.5 Testing and Demonstration

Testing focused on the principal application functions: starting and stopping the webcam, displaying live detections, recognizing objects in the camera frame, capturing a snapshot and recording annotated video. The supplied screenshots demonstrate a blank interface before detection, a bottle detection, and a scene containing both a bottle and a mobile phone.

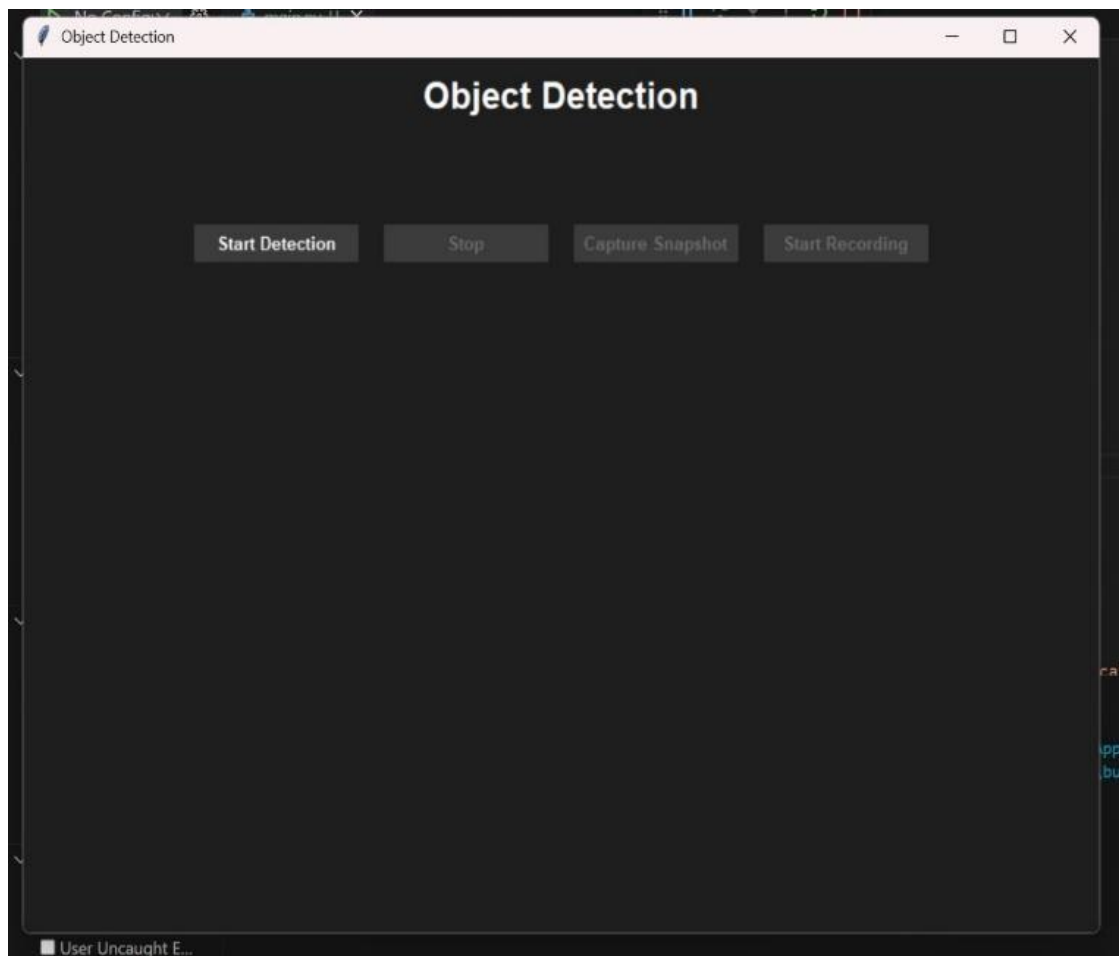


Figure 4.8.2 — Object Detection interface before live detection

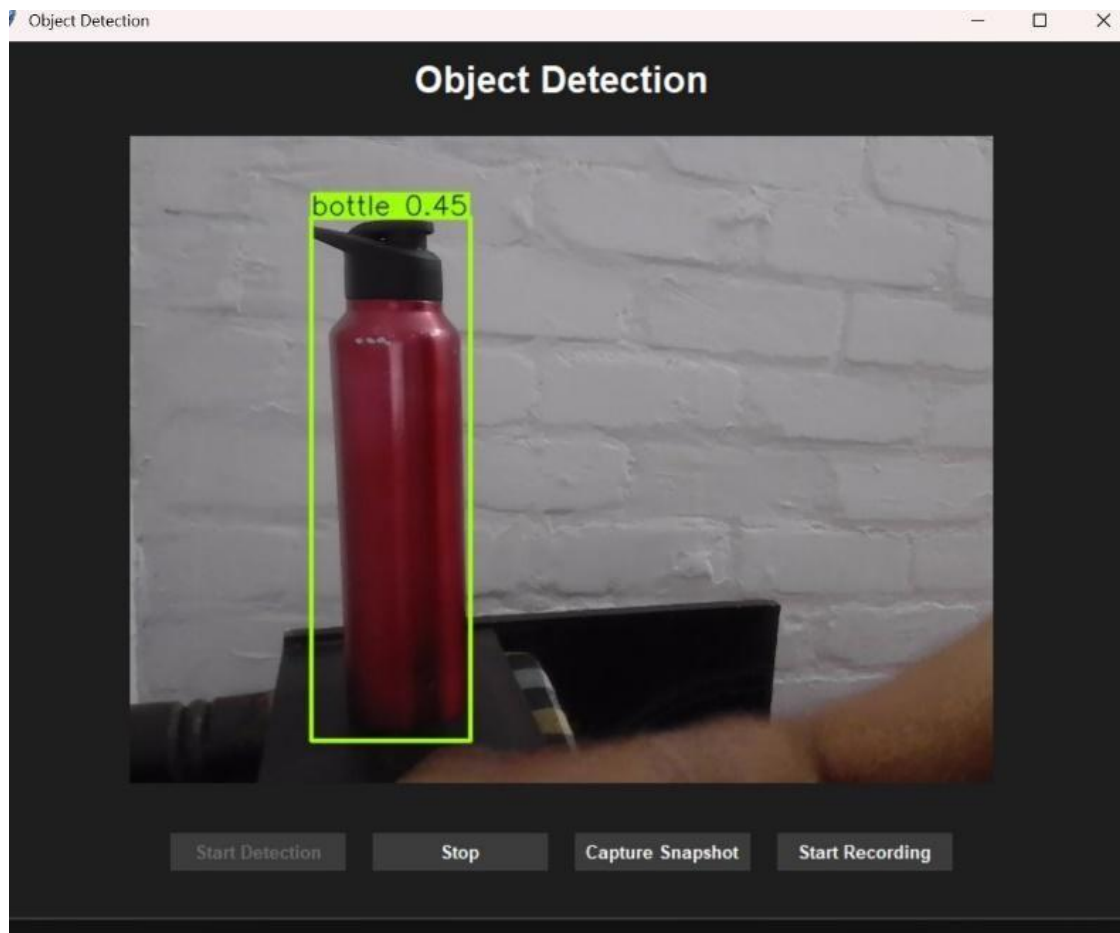


Figure 4.8.3 — YOLOv8 detecting a bottle with a bounding box

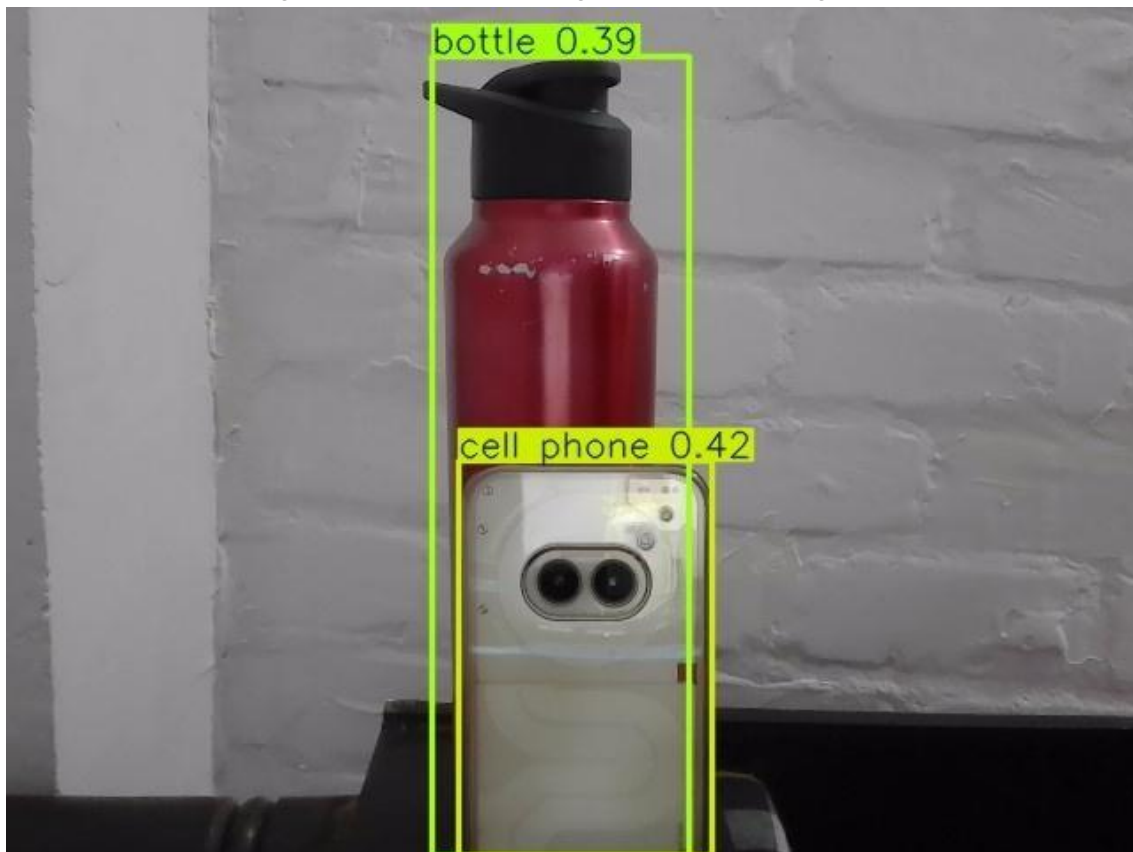


Figure 4.8.4 — Multiple detections: bottle and cell phone

The screenshots provide a visual demonstration of the implemented interface and the model's ability to place class labels and confidence values over detected objects. The project source does not provide a formal accuracy benchmark, dataset size, precision/recall table or FPS measurement; therefore, this report does not invent numerical performance results.

4.8.6 Challenges Faced

- Maintaining real-time performance while performing neural-network inference on each video frame.
- Balancing detection accuracy with computational requirements.
- Handling different lighting, viewing angles, occlusions and cluttered backgrounds.
- Keeping the GUI responsive while processing a continuous camera stream.
- Managing video writer resources and camera release correctly when detection stops.

4.9 Expected Outcomes

The expected outcome is a working desktop application capable of real-time object detection from a webcam, with detected objects visualized through bounding boxes and class labels. The system also provides practical controls for stopping detection, taking snapshots and recording annotated video. The project demonstrates how Python and its computer vision ecosystem can simplify the development of an interactive deep-learning application.

- Real-time object localization and classification.
- Interactive GUI for starting and controlling detection.
- Annotated visual output with bounding boxes and labels.
- Snapshot and recording capabilities.
- A foundation for future deployment and model optimization.

4.9.1 Future Scope

- Improved algorithms and architectures that provide higher accuracy with lower computational cost, including transformer-based approaches such as DETR.
- Better multi-object and multi-class detection, especially for small, overlapping or partially occluded objects.
- Edge and mobile deployment using quantization, pruning and lightweight models such as MobileNet.
- Integration with augmented reality applications for real-time interaction with physical objects.
- Self-supervised and few-shot learning to reduce dependence on large labelled datasets.
- Improved robustness under poor lighting, weather variation and highly dynamic environments.
- Application-specific models for pedestrians, vehicles, retail products or wildlife monitoring.

4.9.2 Conclusion

The Object Detection Using Python project demonstrates the practical integration of deep learning and computer vision into a real-time desktop application. Using YOLOv8 through Ultralytics, OpenCV for video processing and Tkinter for the user interface, the system can detect objects in a live camera feed and display their locations using bounding boxes and class labels.

The project also illustrates the importance of model selection, preprocessing, inference speed, post-processing and user-interface design in a real-time computer vision system. While the current implementation is a useful academic prototype, future improvements can focus on performance benchmarking, custom datasets, model optimization and deployment to mobile or edge platforms.

4.10 References

1. OpenCV-Python Documentation — https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
2. Python Official Documentation — <https://docs.python.org/3/>
3. PyImageSearch — Tutorials on Object Detection and OpenCV — <https://pyimagesearch.com>
4. Ultralytics Documentation — YOLO and Ultralytics Python interface — <https://docs.ultralytics.com/>
5. The supplied Minor Project Report: Object Detection Using Python, IILM University, School of Computer Science and Engineering.

--- End of Report ---

Ref No: C17Y2026M061447325

Internship Offer Letter

Student Name : Ravikant Mishra
Stream : B.Tech
Branch : B.Tech(Computer Science and Engineering)
Institute Name : IILM University, Greater Noida
Internship Domain : Python Fullstack Developer
Internship Duration : June 2026 - August 2026
AICTE Student ID : STU698af9f6d6e0d1770715638

We are pleased to inform you that you have been selected for the 8-Week **AICTE – EduSkills Virtual Internship Program**.

During this internship, you will learn and demonstrate industry-relevant skills that will enhance your employability and strengthen your confidence in the subject area.

The program is designed to provide structured learning combined with practical exposure.

Internship Structure & Evaluation:

- The internship will follow a **week-wise structured learning plan**.
- You will be required to complete **weekly assessments** to track your progress and understanding.
- Certain modules will require submission of **project documentation and assignments**.
- In the final week, you must appear for a **Final Assessment Test**.
- The final grade will be based on your overall performance, including weekly assessments, project submissions, and the final test.

Upon successful completion of all required steps and assessments, you will be awarded a **Virtual Internship Certificate**.

Please note that this offer letter will be considered valid only after the successful completion of the internship program and fulfillment of all assessment requirements, including obtaining the final completion certificate. Without the final completion certificate, this offer letter shall not be treated as a valid document for any official purpose.

